



CS 498: Quantization

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

- H2O: not all KV cache (tokens) are equally important
 - Keep the ones with high attention score to cap memory usage
- Attention Sink:
 - Keep the first several tokens and recent tokens
 - Hard to interpret

Inference Speedup

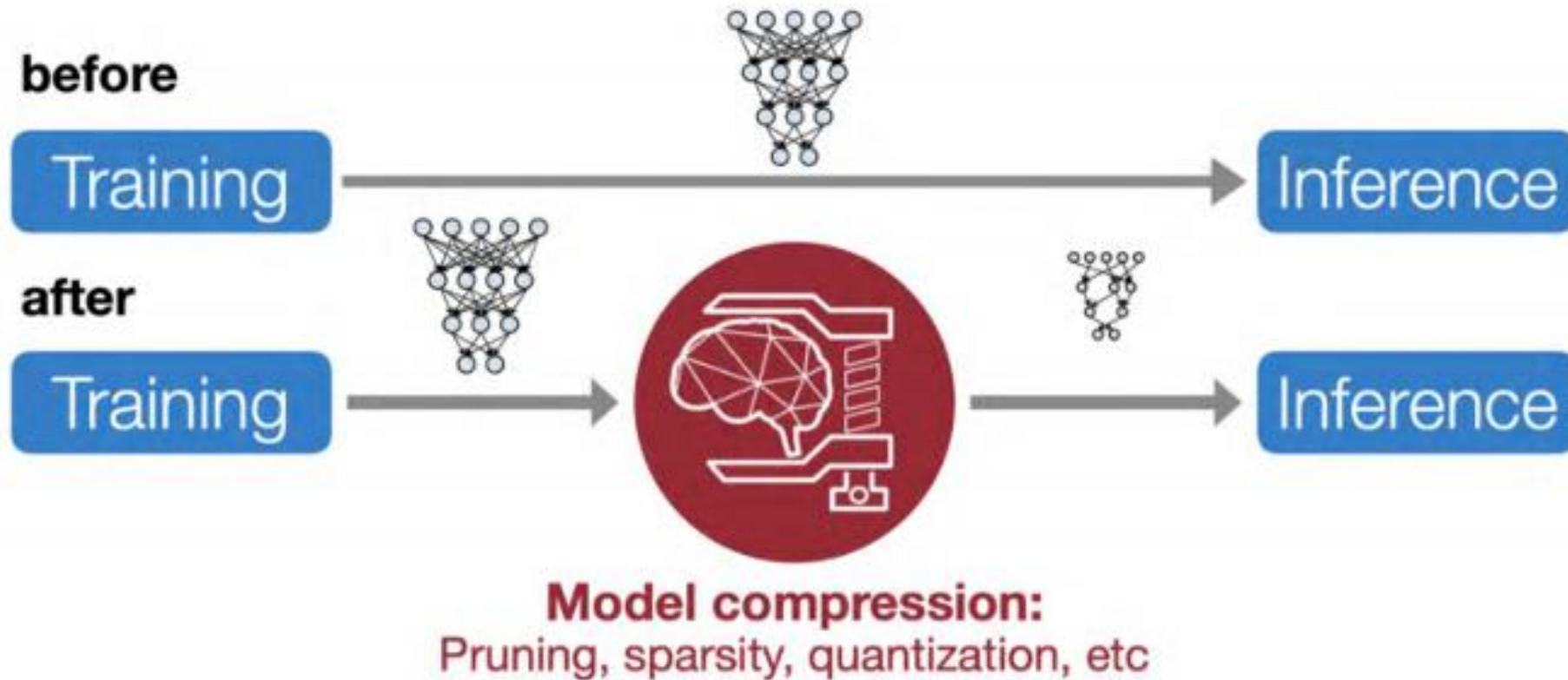
- LLM Quantization

Objective:

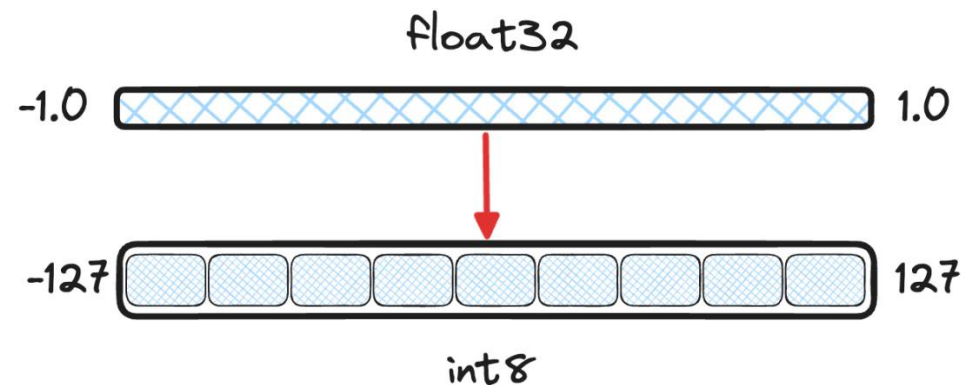
- Understand how LLM quantization reduces memory and compute cost
- Learn the core ideas behind SmoothQuant
- Analyze the trade-offs in accuracy and efficiency introduced by quantization

Panel Discussion: AWQ (MLSys'24 Best Paper Award)

Bridge the Gap through Model Compression



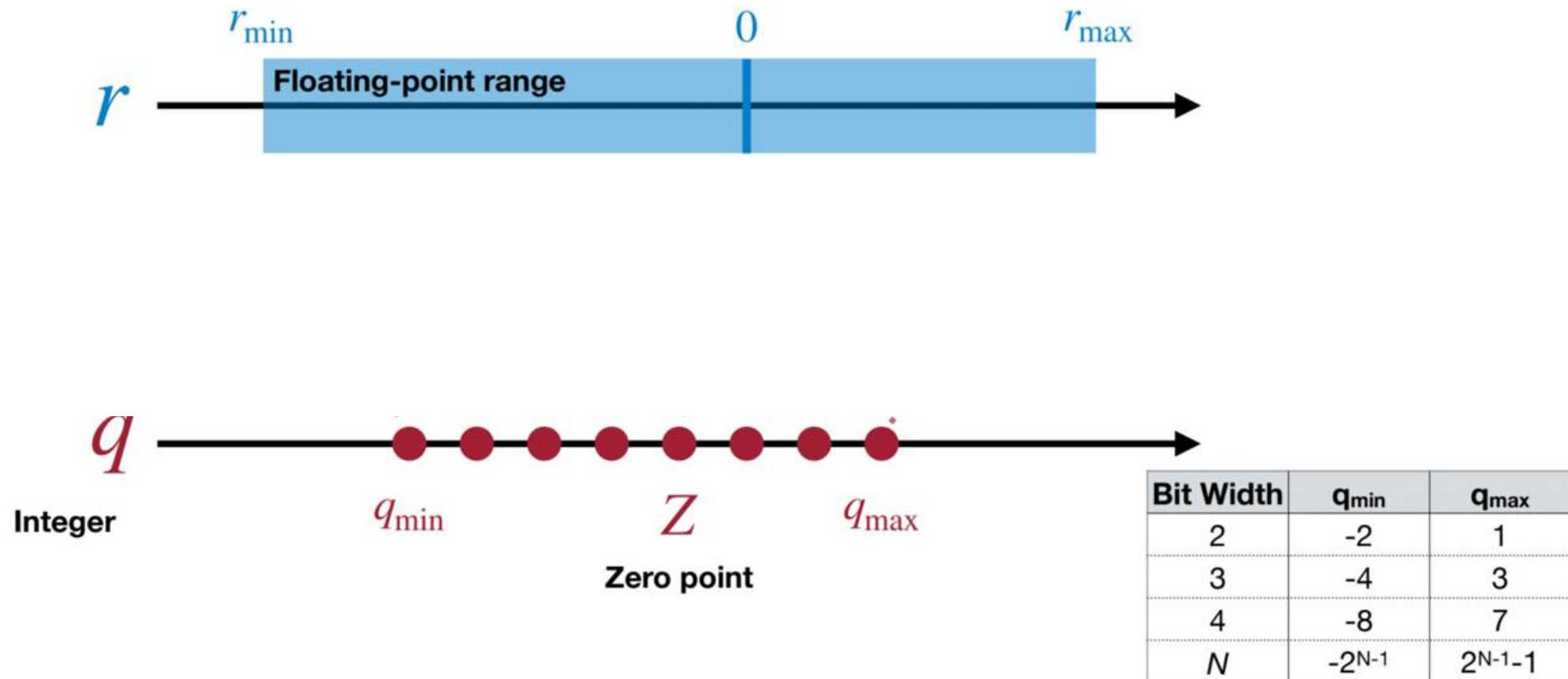
- Reduce the bits per weight, saving memory consumption
- Accelerate inference speed on supporting hardware



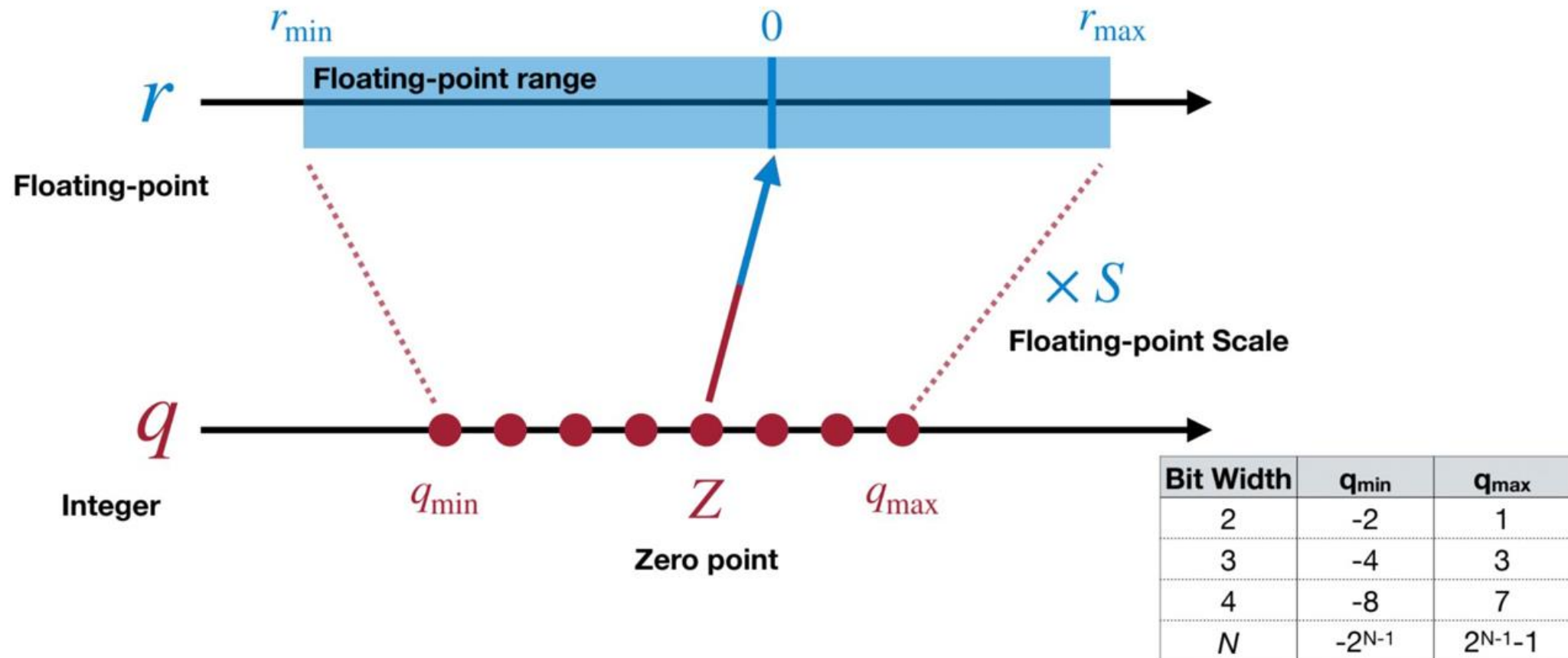
Key Concepts: Linear Quantization + Round-to-Nearest



Key Concepts: Linear Quantization + Round-to-Nearest



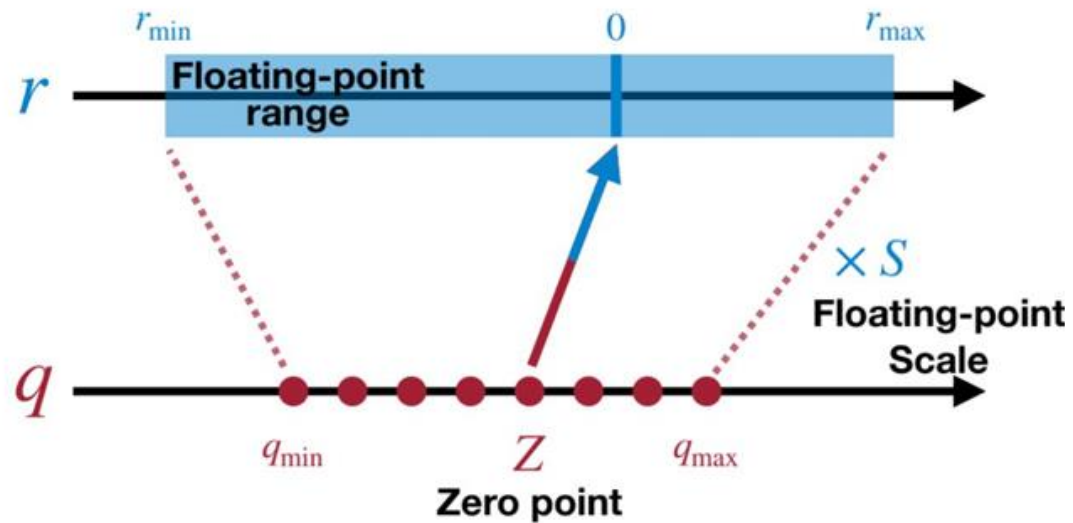
- An affine mapping of integers to real numbers $r = S(q - Z)$



Asymmetric vs. Symmetric Linear Quantization



Full range mode



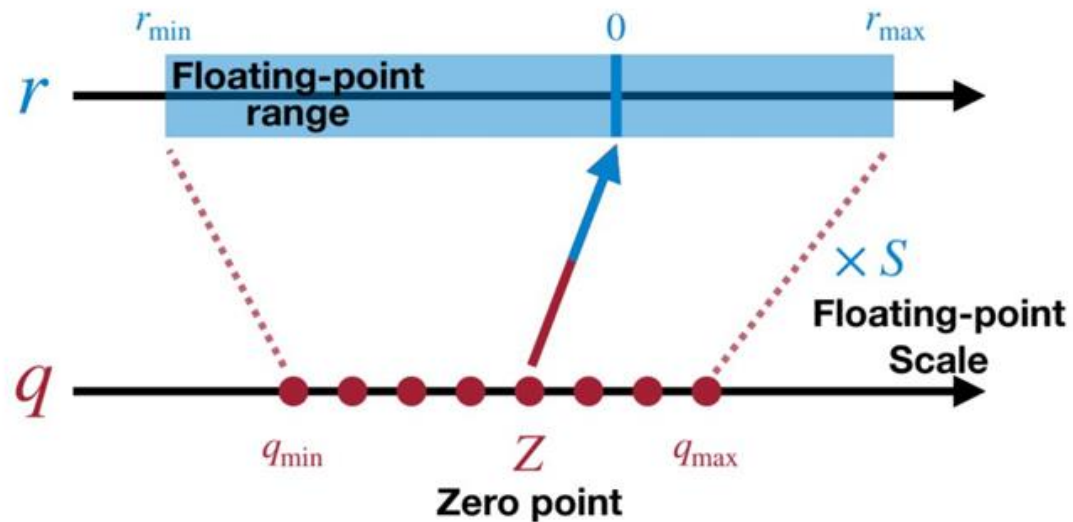
Question: How do we calculate the scaling factor S ?

Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Asymmetric vs. Symmetric Linear Quantization



Full range mode



$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

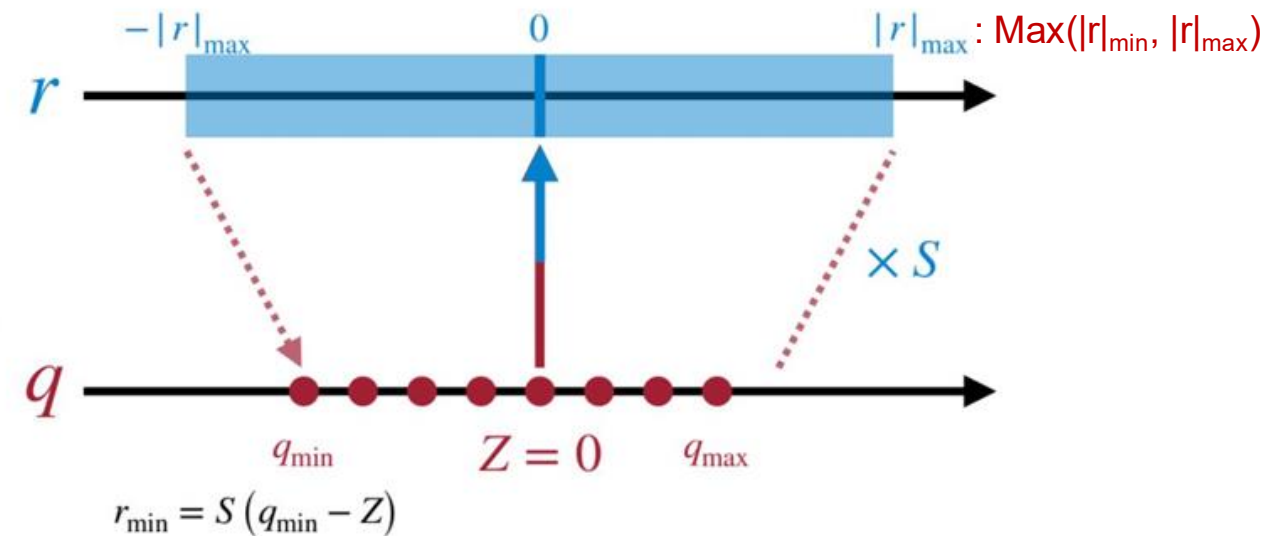
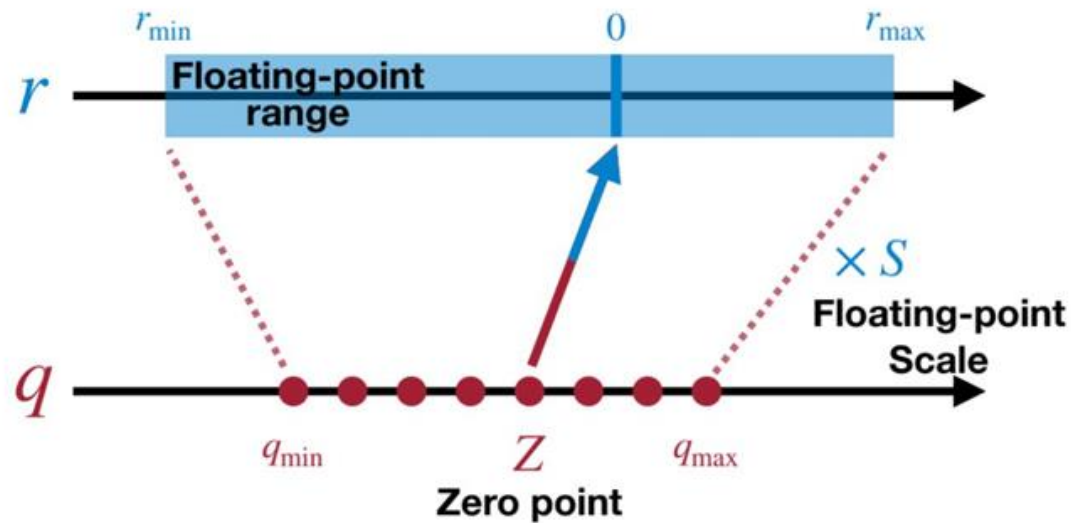
Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Question: How do we calculate the scaling factor S ?

Asymmetric vs. Symmetric Linear Quantization



Full range mode



$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

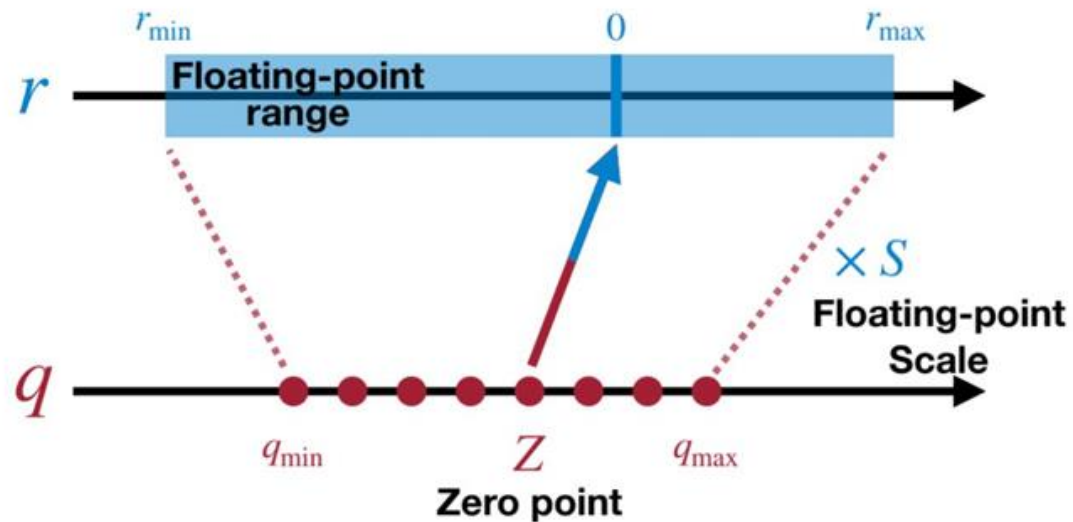
Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Question: How do we calculate the scaling factor S?

Asymmetric vs. Symmetric Linear Quantization

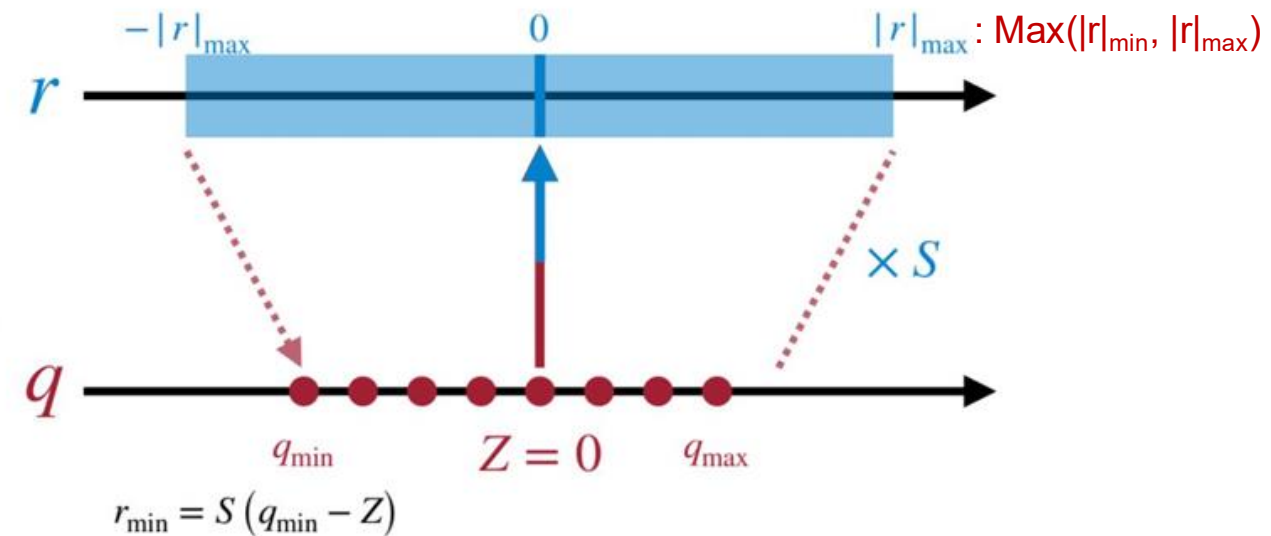


Full range mode



$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

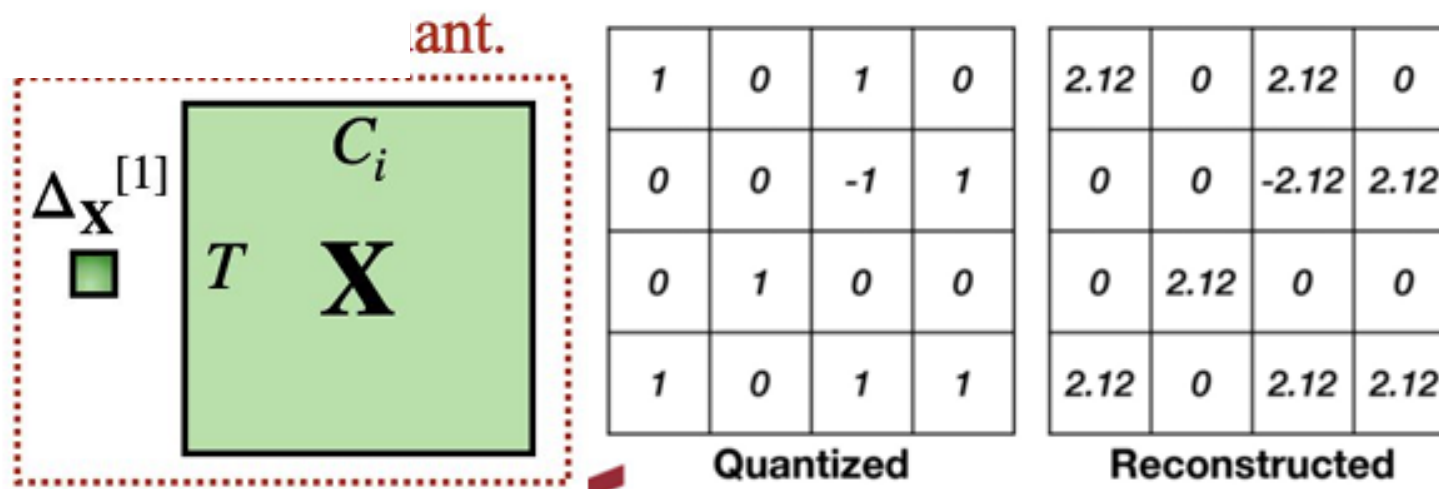


$$S = \frac{r_{\min}}{q_{\min} - Z} = \frac{-|r|_{\max}}{q_{\min}} = \frac{|r|_{\max}}{2^{N-1}}$$

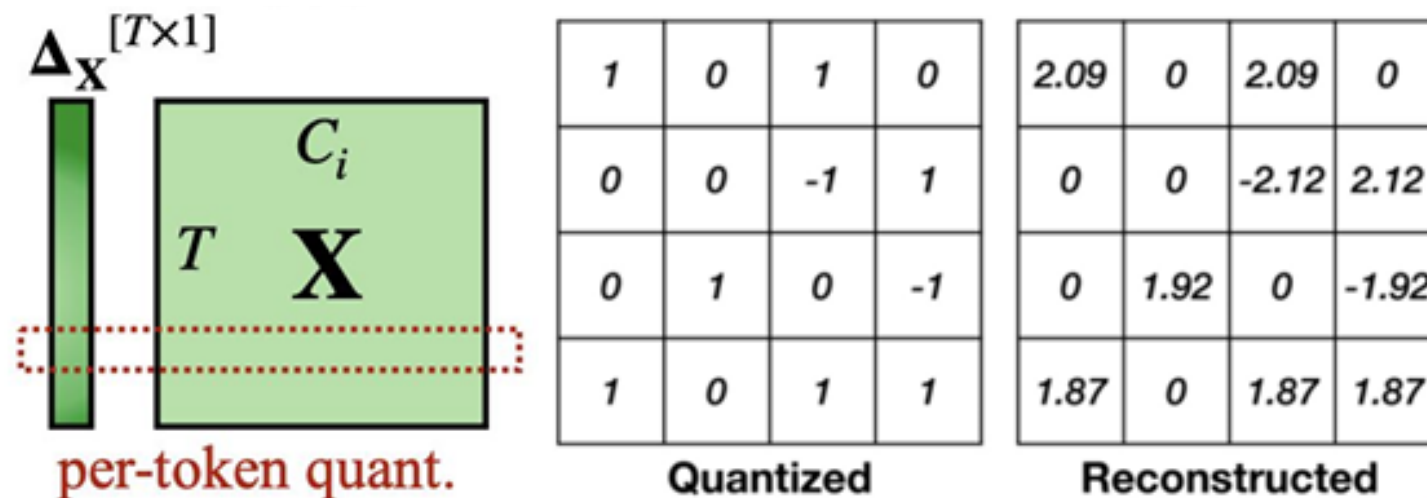
- use full range of quantized integers
- example: PyTorch's native quantization,

- Per-Tensor Quantization
 - Uses a single scaling factor for the entire matrix

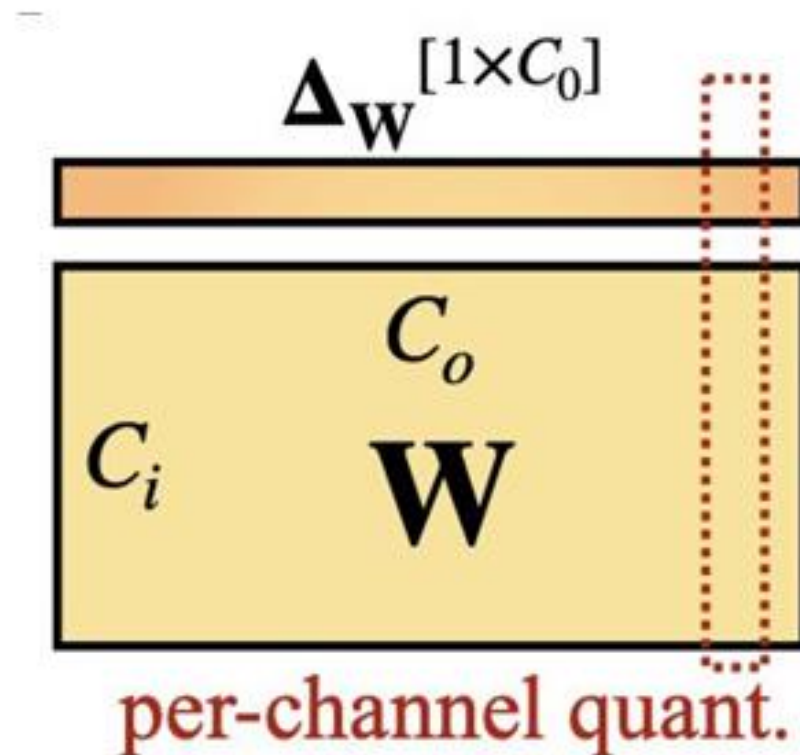
$$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$$



- Per-Tensor Quantization
- Per-Token Quantization
 - Uses different scaling factor for activations associated with each token
- Per-Channel (e.g., weight column) Quantization
- Group-wise Quantization



- Per-Tensor Quantization
- Per-Token Quantization
- Per-Channel (e.g., weight column) Quantization
 - Uses different scaling factors for each weight channels
- Group-wise Quantization

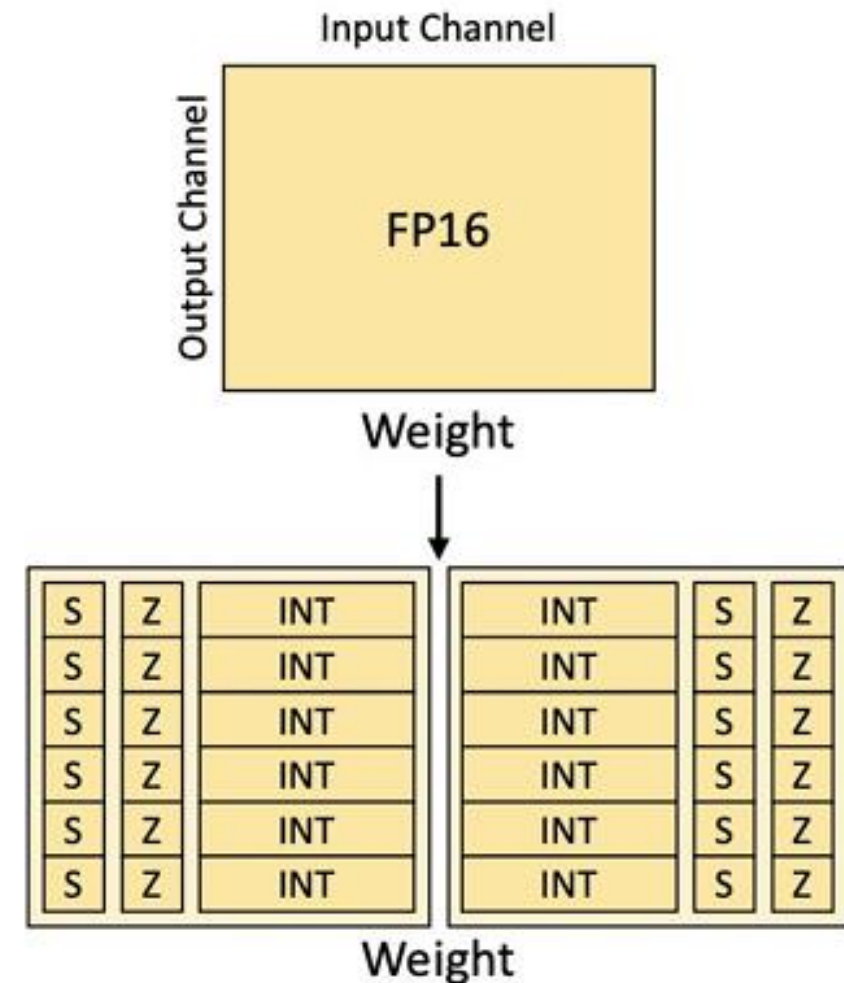


- Per-Tensor Quantization
- Per-Token Quantization
- Per-Channel (e.g., weight column) Quantization
 - Uses different scaling factors for each weight channels
- Group-wise Quantization

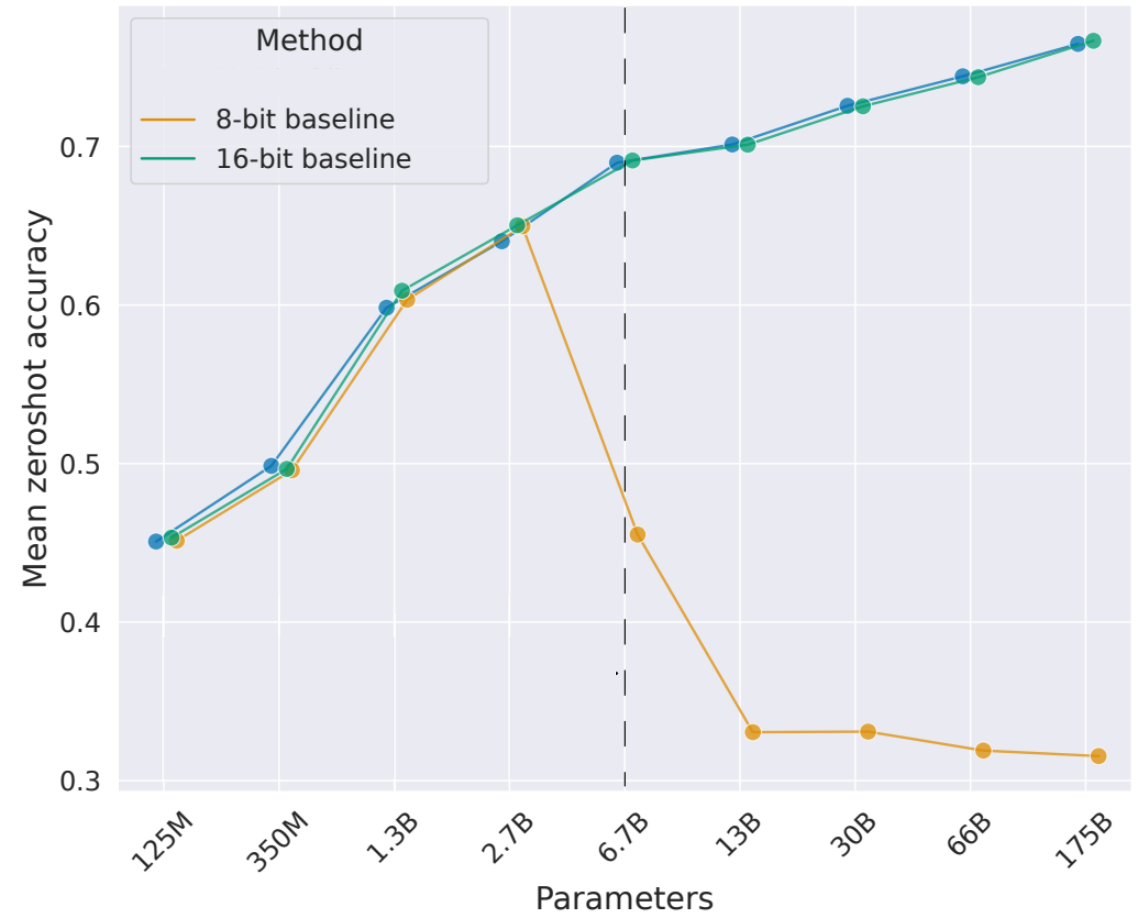
Model size (OPT-)	6.7B	13B	30B	66B	175B
FP16	64.9%	65.6%	67.9%	69.5%	71.6%
INT8 per-tensor	39.9%	33.0%	32.8%	33.1%	32.3%
INT8 per-token	42.5%	33.0%	33.1%	32.9%	31.7%
INT8 per-channel	64.8%	65.6%	68.0%	69.4%	71.4%

Per-channel QAT achieves best acc (tokens often demonstrate similar range patterns in the same channel). but hard to support in hardware

- Per-Tensor Quantization
- Per-Token Quantization
- Per-Channel (e.g., weight column) Quantization
- Group-wise Quantization
 - Different scaling factors for different channels groups



- Standard quantization strategy leads to catastrophic accuracy drop



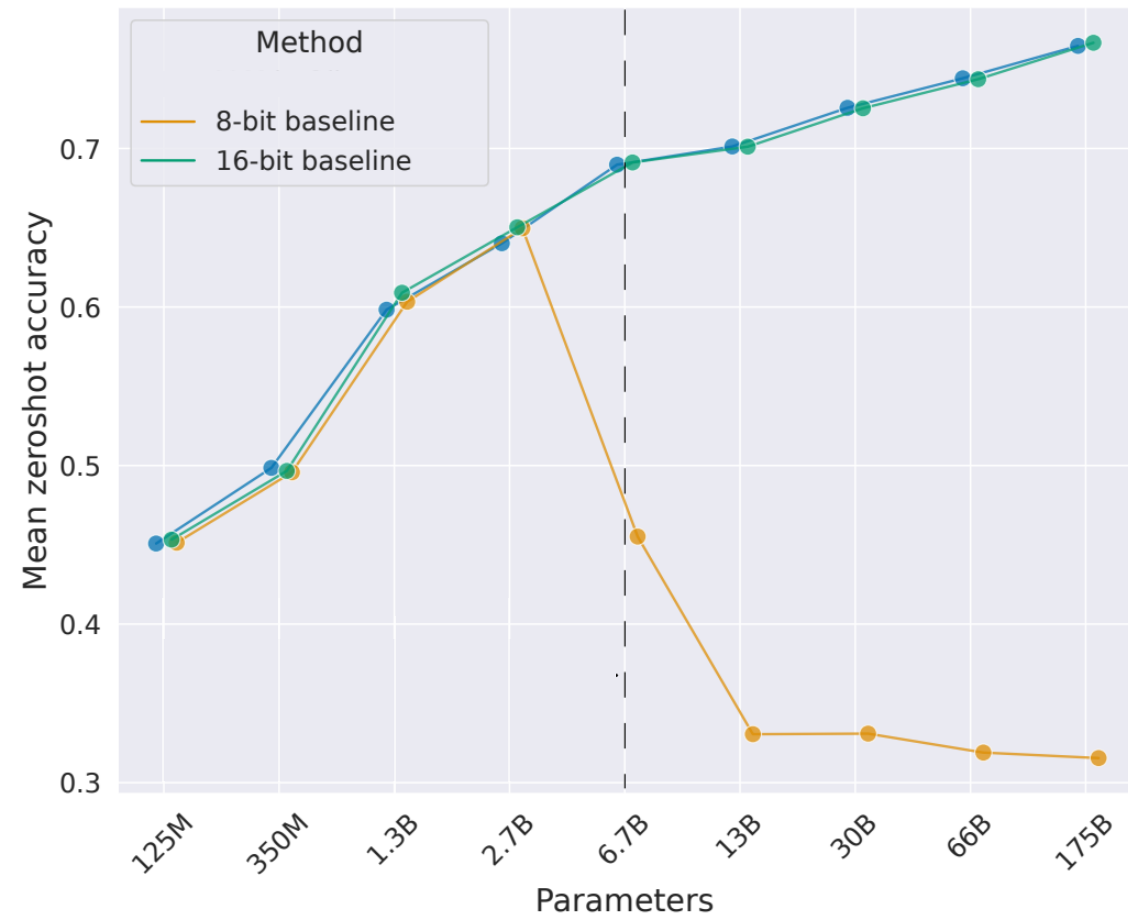
- Standard quantization strategy leads to catastrophic accuracy drop

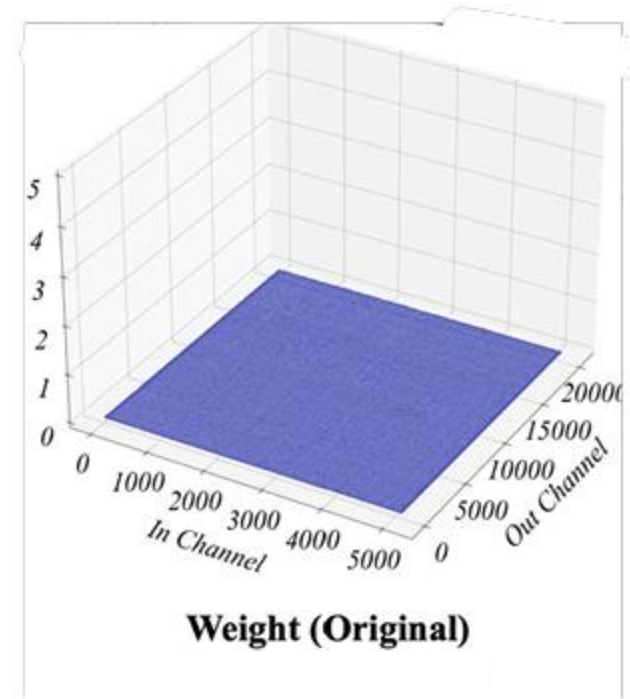
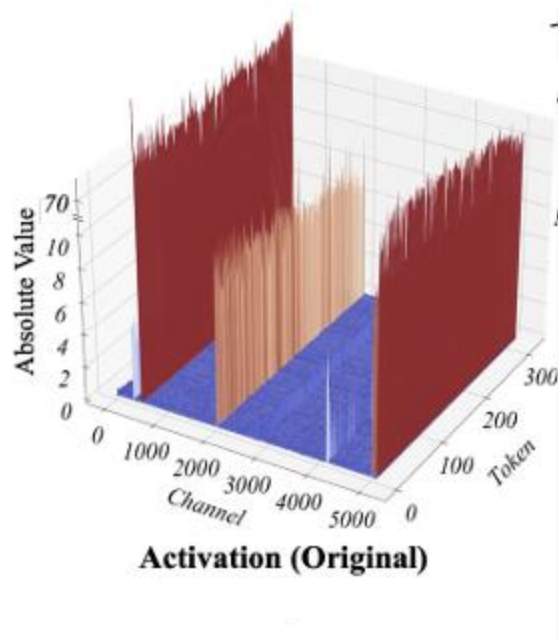
Output = Activation X Weight

Precision	Lambada (↑)
W16A16	49.3
W8A16	49.3
W16A8	44.7
W8A8	42.6

WxAy means x-/y-bit for weight/activation.

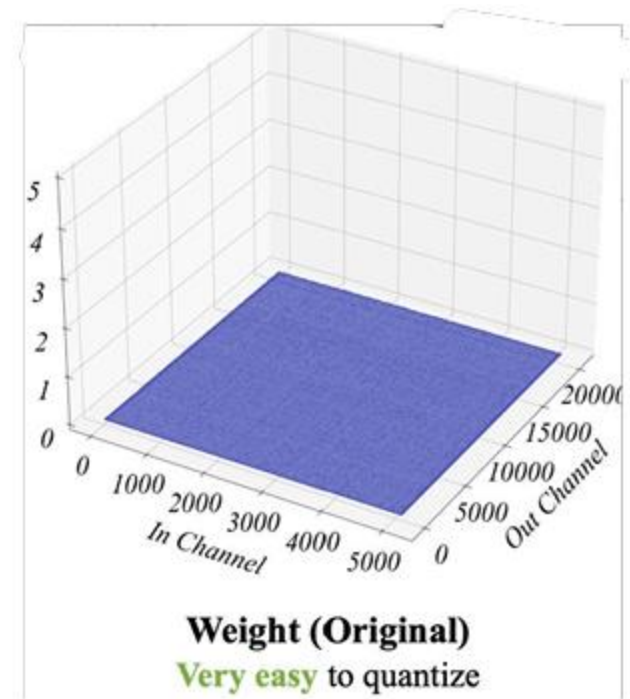
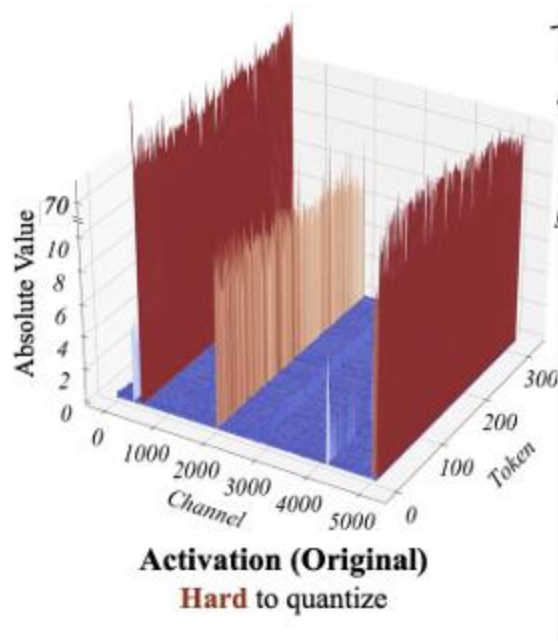
INT8 activation quantization causes the primary accuracy loss



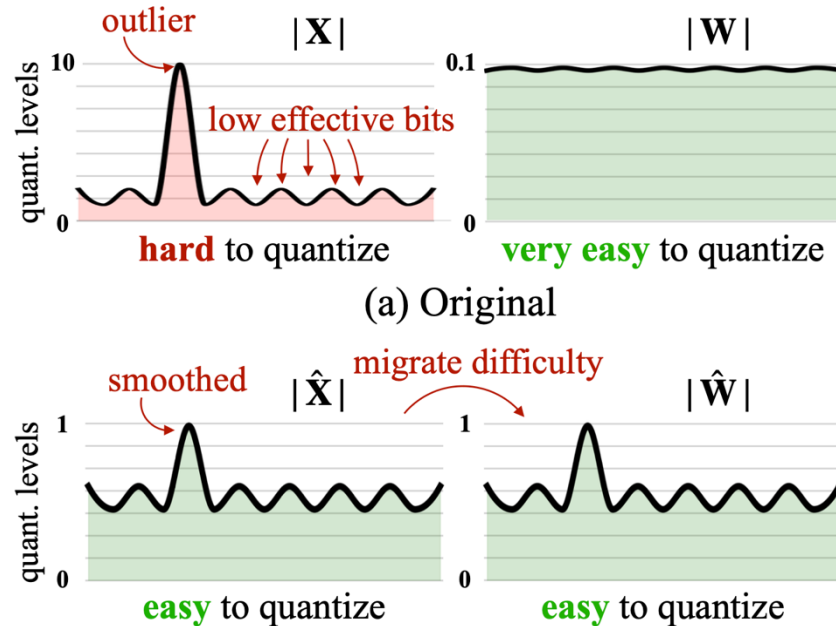


- Output = Activation X Weight, so we want to quantize both weights and activations into INT8 (i.e., W8A8) to utilize the integer kernels (e.g., INT8 GEMM)
- Activation has many outliers (~70x larger) while the weight value is smooth (flat)
 - Outliers leave few effective bits for most values

Activations are Intrinsically Challenging to Compress



- Output = Activation X Weight, so we want to quantize both weights and activations into INT8 (i.e., W8A8) to utilize the integer kernels (e.g., INT8 GEMM)
- Activation has many outliers (~70x larger) while the weight value is smooth (flat)
 - Outliers leave few effective bits for most values



$$\mathbf{Y} = (\mathbf{X} \text{diag}(\mathbf{s})^{-1}) \cdot (\text{diag}(\mathbf{s}) \mathbf{W}) = \hat{\mathbf{X}} \hat{\mathbf{W}}$$

$$\text{e.g., } \mathbf{Y} = \mathbf{XW} = (0.01\mathbf{X})(100\mathbf{W})$$

- Key idea: Migrating the quantization difficulty from activation to weight

$$\mathbf{Y} = (\mathbf{X}\text{diag}(\mathbf{s})^{-1}) \cdot (\text{diag}(\mathbf{s})\mathbf{W}) = \hat{\mathbf{X}}\hat{\mathbf{W}}$$

- Push all quantization difficulty from activations to weights (all channels have same maximum magnitude). To increase effective quantization bits for channels

$$\mathbf{s}_j = \max(|\mathbf{X}_j|), j = 1, 2, \dots, C_i,$$

$$\mathbf{Y} = (\mathbf{X} \text{diag}(\mathbf{s})^{-1}) \cdot (\text{diag}(\mathbf{s}) \mathbf{W}) = \hat{\mathbf{X}} \hat{\mathbf{W}}$$

- Push all quantization difficulty from activations to weights (all channels have same maximum magnitude). To increase effective quantization bits for channels

$$\mathbf{s}_j = \max(|\mathbf{X}_j|), j = 1, 2, \dots, C_i,$$

- Push all quantization difficulty from weights to activations:

$$\mathbf{s}_j = 1 / \max(|\mathbf{W}_j|).$$

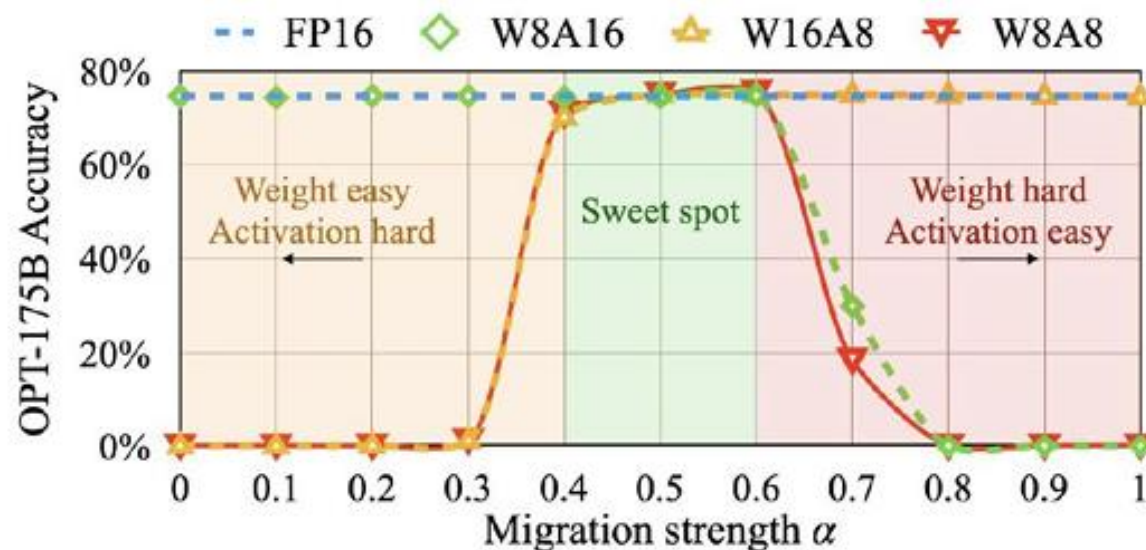
- Balance difficulty according to α :

$$\mathbf{s}_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}$$

Choosing α



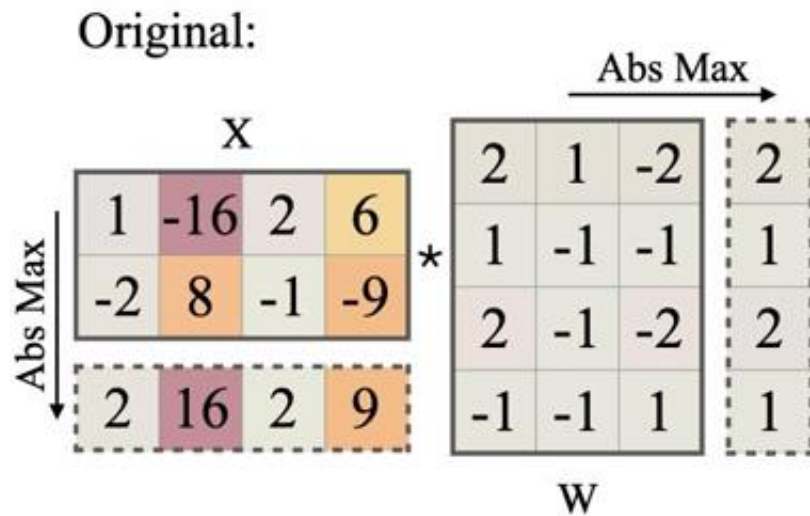
- Case-by-case decision $s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}$
- If α is too large, weights will be hard to quantize. If too small, activations will be hard to quantize.
- Goal: Make activations and weights both easy to quantize.



Example of SmoothQuant



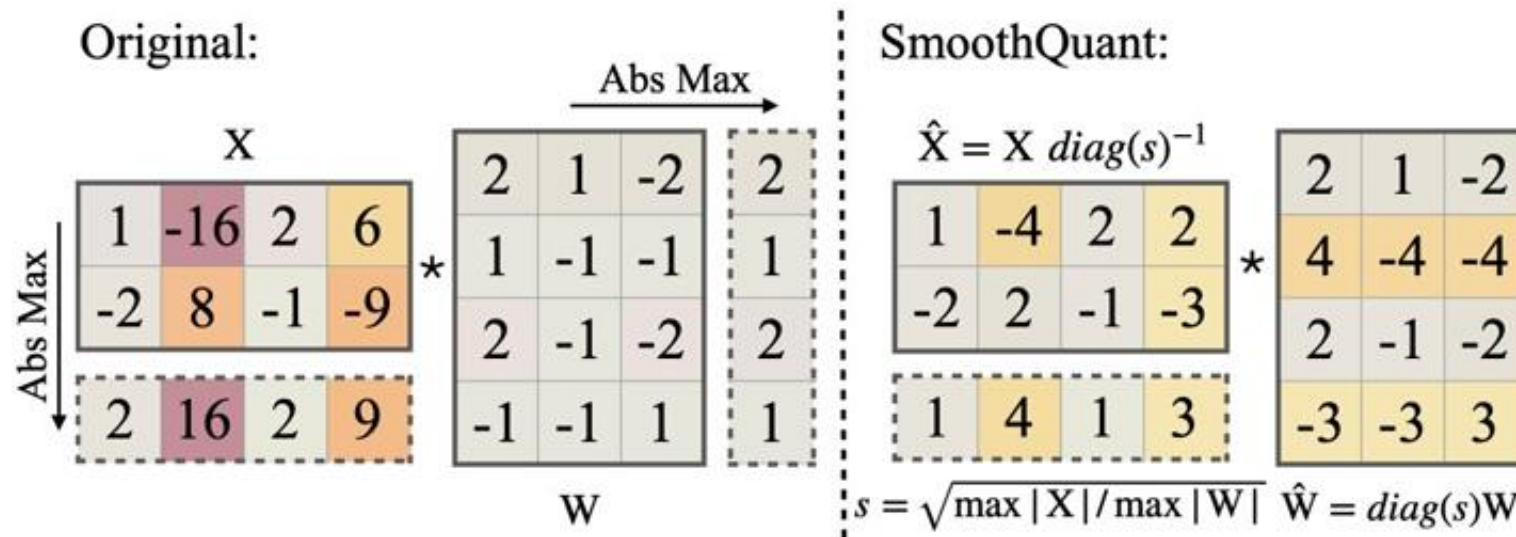
- 1) Applying Smoothing Factor
- 2) Quantize (constant step size)



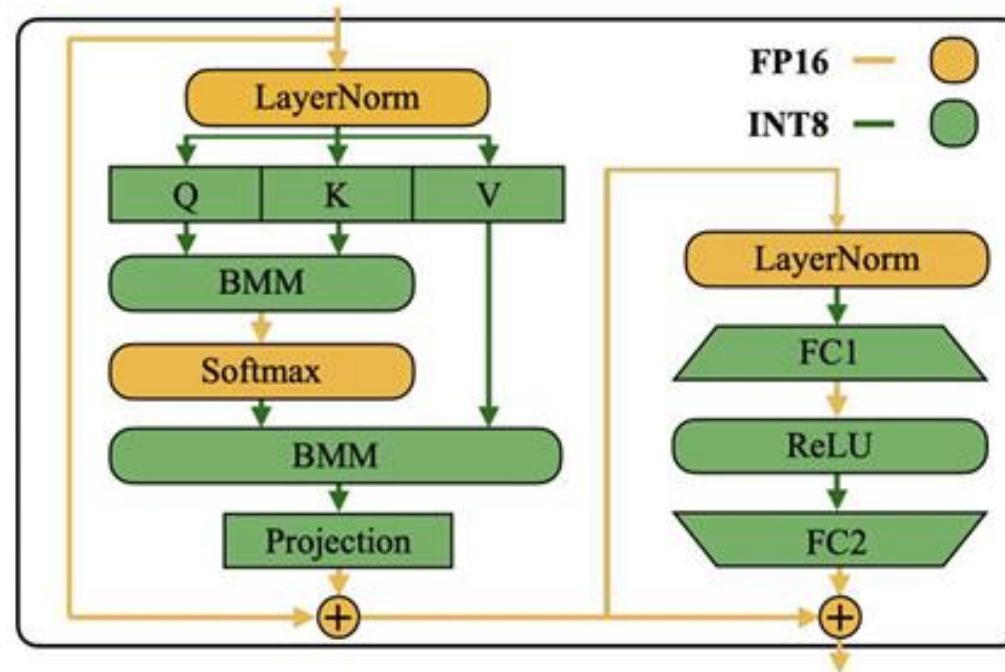
Example of SmoothQuant



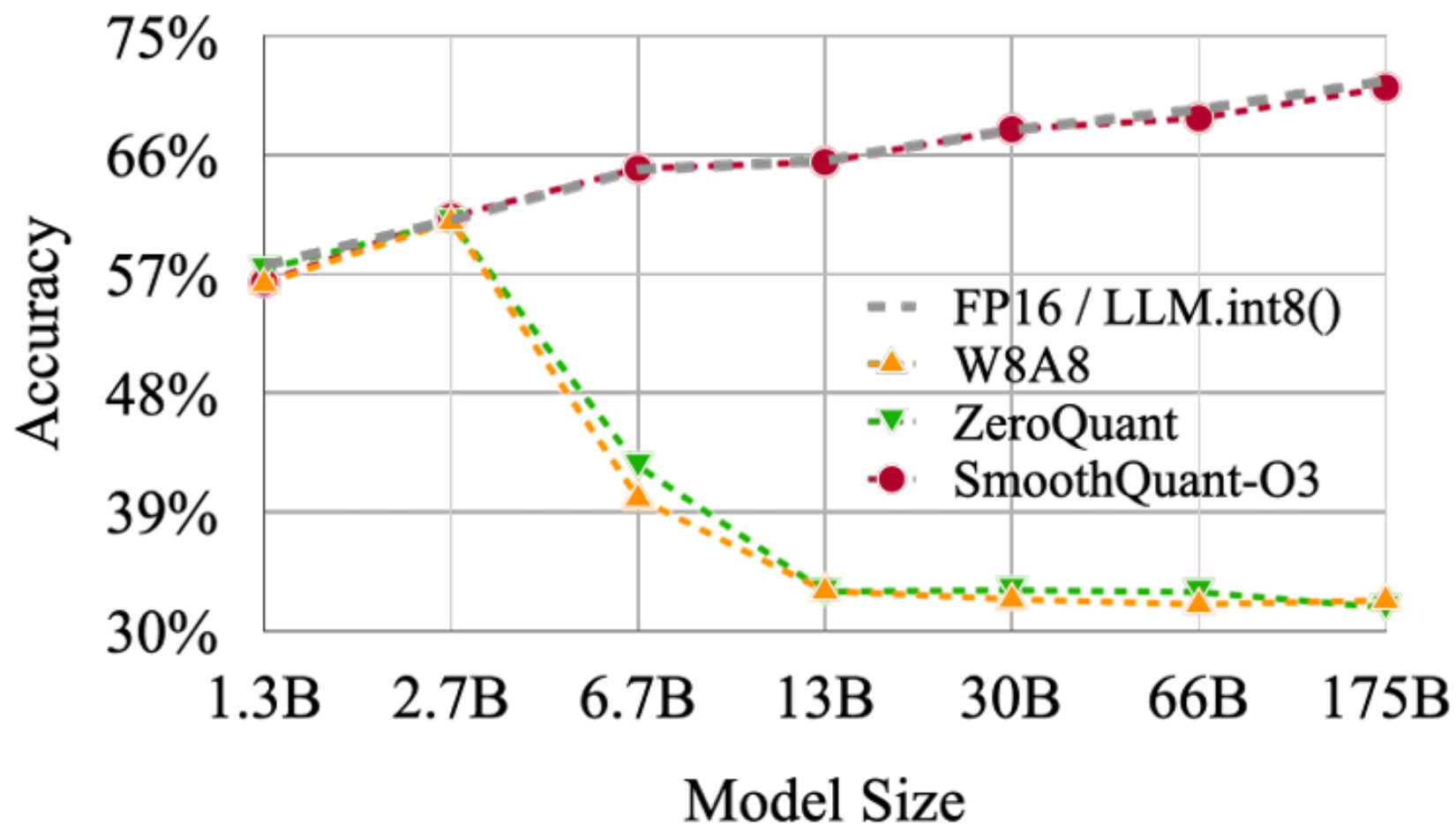
- 1) Applying Smoothing Factor
- 2) Quantize (constant step size)



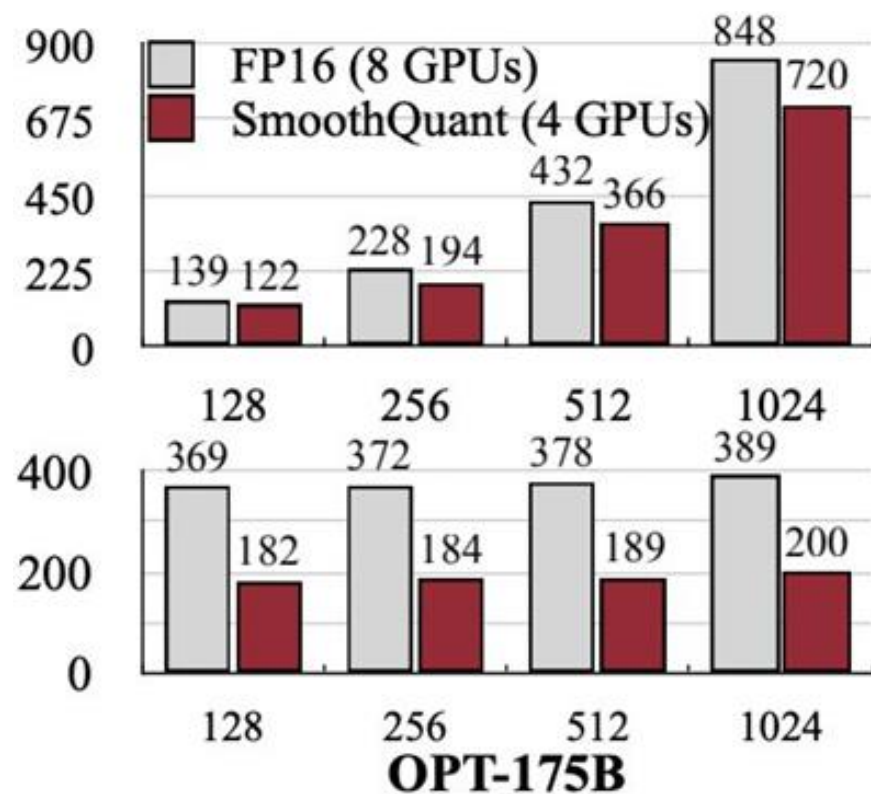
- Applying SmoothQuant to transformers
 - Compute and memory intensive ones in INT8
 - Lightweight operators (e.g., Softmax) in FP16



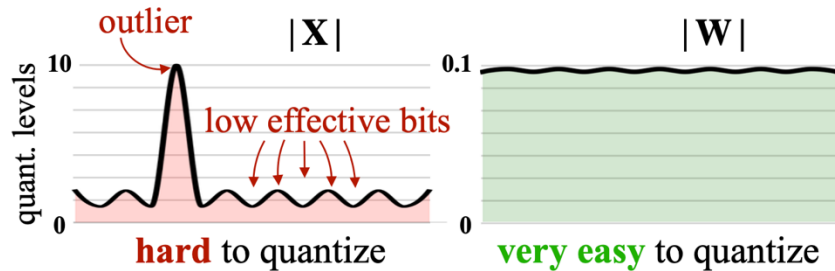
Evaluation: OPT 1.3B to 175B



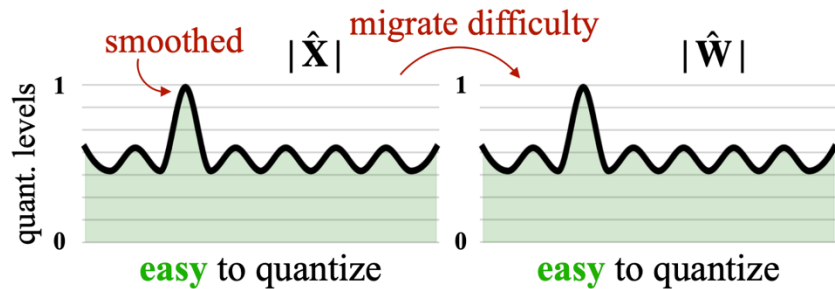
- Similar or faster latency with half #GPUs



Integrated into TensorRT and vLLM



(a) Original



- Limitations:

- Calibration dependency (e.g., smoothing factor)
- Does not really eliminate outliers

AWQ: ACTIVATION-AWARE WEIGHT QUANTIZATION FOR ON-DEVICE LLM COMPRESSION AND ACCELERATION

Ji Lin^{*1} Jiaming Tang^{*12} Haotian Tang^{†1} Shang Yang^{†1} Wei-Ming Chen³ Wei-Chen Wang¹
Guangxuan Xiao¹ Xingyu Dang¹⁴ Chuang Gan⁵⁶ Song Han¹³
<https://github.com/mit-han-lab/llm-awq>

Identify salient model weights to apply QAT based on activations.

$$\text{Objective: } \text{Min} \mathcal{L}(\mathbf{s}) = \|Q(\mathbf{W} \cdot \text{diag}(\mathbf{s}))(\text{diag}(\mathbf{s})^{-1} \cdot \mathbf{X}) - \mathbf{W}\mathbf{X}\|$$